Data collection guide: Go2 gesture dataset

You will record videos of people gesturing near the robot dog, label every gesture, check the data, and hand it to Himasara. No coding is needed; you only type the commands shown here. If something breaks, see **section 13**.

0. What this is about (2 minutes)

A robot dog is controlled by hand gestures. In a crowd, it must only obey the person who is actually gesturing **to the dog**, and ignore people who gesture **to each other**. To test this, we need recordings where:

- one person (the **commander**) gestures to the dog, and
- other people (the **bystanders**) gesture to each other near the dog.

The software records everything. Afterwards, two people watch each gesture and mark whether it was meant for the dog or not.

What you hand over: about **36 recorded takes** with labels from **2 people**, one report file, and the paper forms.

Suggested schedule:

Day	Work	Time
Day 0	Setup and tests (sections 1–5)	2–3 h
Days 1–3	One recording session per day (sections 6–9), labelling that evening (section 10)	about 2 h + 1.5 h per day
Day 4	Check and hand over (sections 11–12)	1 h

---

1. What you need

- **The laptop** with Ubuntu 22.04/24.04, at least **40 GB free disk**, charger.
- **Orbbec camera**, its **USB-3 cable** (blue port on the laptop), and a way to mount it firmly on the **front of the dog** (bracket or strong tape). Point it straight forward, level.
- **Unitree Go2** (charged) and its **remote or the Unitree app**, for driving.
- **Tape measure** and **masking tape** for floor marks.
- **Printed forms** (`forms_to_print.pdf`): consent forms, the participant sheet and the take log.
- **People:** 4 per session (1 commander + 3 bystanders), and **8–10 different people** over the 3 days. Everyone takes a turn as commander.
- **A portable SSD / USB drive (64 GB+)** for the handover.

2. Get the code (once)

Himasara will add you to the GitHub repository. Then, in a terminal:

```
cd ~
git clone <the repo link Himasara sends you> go2-engagement
cd go2-engagement
git checkout -b data-collection
```

(If you got a zip instead: unzip it into your home folder as `go2-engagement`.)

3. Set up the laptop (once, about 15 min)

```
cd ~/go2-engagement/engage
bash setup.sh
```

It asks for the laptop password. At the end it should print `imports + models OK` and `Setup done`.

Every time you open a new terminal for this project, first run:

```
cd ~/go2-engagement/engage
source .venv/bin/activate
```

You'll see `(.venv)` at the start of the line. All commands below assume this.

4. Test the camera (5 min)

Plug the Orbbec into a **blue USB-3 port**. If you just ran setup, unplug and re-plug it once.

```
python check_orbbec.py
```

You should see the colour image on the left and a coloured depth image on the right. Stand **2 m** in front of it: the "centre" number should read about **2.0 m**. Press `q` to close.

If it fails, see section 13.

5. Test the full system (10 min)

```
python run_live.py --source orbbec --take test_1 --scenario test
```

Stand 2 m in front of the camera. You should see a box around you with numbers. Do these checks:

1. Face the camera and show a **fist**. The box turns **green** and says `ACCEPT`.
2. Turn your body sideways, look at a wall and show a fist. The box should stay **orange** (seen but not accepted).
3. Ask a second person to stand next to you. Both of you should have boxes.

The top line must say `depth`, not `NO DEPTH`. Press `q` to stop. Takes named `test_...` are only for testing.

The 5 gestures (practise these with every participant)

Gesture	How to make it	Meaning
Palm	Open hand, fingers spread, palm toward the target	STOP
Fist	Closed fist toward the target	Forward
Call	Thumb and little finger out, like a phone	Backward
One	Index finger pointing up	Turn left
Two-up	Index and middle finger up, held together	Turn right

Hold each gesture **about 2 seconds**, with the hand **beside or in front of the body** (not hidden behind someone), then drop the hand.

---

6. Safety rules (read to everyone at the start of every session)

- One person is the **safety driver**. They hold the dog's remote the whole time and stop the dog at once if anything looks wrong.
- The dog moves **slowly only**. Keep at least **1 m** between the dog and people while it walks.
- Clear the floor of bags and cables. No children in the area.
- In walking takes, the **laptop carrier** walks close behind the dog so the USB cable never pulls. Tape the cable along the dog's body.
- The software **never drives the dog** in this collection. Don't use `--drive`.

7. Before each session (15 min)

1. **Consent:** every person signs a consent form before being recorded.
2. **IDs:** write each person on the participant sheet with an ID (P01, P02, ...), and use the **same ID** for the same person on every day. Names only go on the paper sheet, **never in any file or take name**.
3. **Floor marks:** put tape on the floor at **1 m, 2 m, 3 m and 4 m** in front of the dog.
4. **Mount check:** the camera is firmly on the dog, pointing forward. Write the camera height from the floor on the take log, once per session.
5. **Calibration take** (1 min): one person stands on each mark (1, 2, 3, 4 m). At each mark they face the dog for 5 s, then turn sideways for 5 s, then turn their back for 5 s. `bash python run_live.py --source orbbec --take D1_CAL --scenario calibration --location lab --people P01 --notes "1-2-3-4 m: front 5s, side 5s, back 5s"` (Use `D2_CAL` on day 2, and so on.)

8. Recording a take

Every take is **60–90 seconds**. Name it `D<day>_<scenario>_r<repetition>`, for example `D1_S2_r1` (day 1, scenario S2, first try).

Stationary take (dog standing still):

```
python run_live.py --source orbbec --take D1_S2_r1 --scenario stationary \
  --location lab --commander P03 --people P01,P03,P05,P07
```

Walking take (the safety driver walks the dog slowly with the remote):

```
python run_live.py --source orbbec --take D1_W1_r1 --scenario walking \
  --location lab --commander P03 --people P01,P03,P05,P07
```

- Start the command, wait until everyone has a box, say **"go"**, run the script, then press `q`.
- **At most 5 people in the picture**, each **fully inside it** (head to knees, not cut off at the left/right edge).
- **Watch the window:** every person should have a box and the top line should say `depth`. If someone has no box, ask them to step 0.5 m closer, away from the edge, or into better light.
- If something goes wrong (wrong script, the dog stopped, someone walked off), just **record it again** with the next repetition number (`_r2`) and write what happened on the take log. You don't need to delete anything.

- `--notes "..."` adds a short comment, for example `--notes "P05 hidden behind P07 half the time"`.
- Write **every take** on the paper take log: name, time, commander, people, anything odd.

9. The 9 scenarios (do each **4 times** over the 3 days, with a **different commander** each time)

Unless the script says otherwise, people stand **1.5–4 m** from the dog and spread out left to right so they don't block each other.

Code	Dog	Script (60–90 s)
S1	still	Commander alone, facing the dog, does all 5 gestures twice. The other people stand at the side, still, hands down.
S2	still	Commander faces the dog and gives a gesture every ~5 s. Bystanders stand in pairs facing **each other** and keep gesturing **to each other** every ~5 s (any of the 5 gestures).
S3	still	Commander and one bystander gesture **at the same moment**: the commander to the dog, the bystander to a friend. Count "3, 2, 1, now" out loud. Repeat ~8 times.
S4	still	Commander turns to a friend, gestures **at the friend**, then turns back and gestures **at the dog**. Repeat ~6 times.
S5	still	Two people face the dog and gesture **to it together** (one palm, one fist). Repeat ~6 times, swapping who does which.
S6	still	Bystanders walk across in front of the dog (1.5–4 m away) while waving or gesturing to someone **off to the side**. The commander stands still and gestures to the dog now and then.
W1	walking	The safety driver walks the dog slowly toward and around the group. The commander gives gestures to the dog; bystanders stand around, hands down.
W2	walking	Like W1, but a bystander near the dog keeps gesturing **to a friend**.
W3	walking	Like W1, but people walk **between** the dog and the commander now and then.

Also include, a few times per session: - **Close-up moments:** the commander at 1–1.5 m, so the camera can see the eyes clearly. - **Looking without gesturing:** a bystander looks straight at the dog for a few seconds with hands down.

Target: about 36 takes in total (9 scenarios × 4). Roughly 12 takes per session.

10. Labelling (each evening, two people separately)

Two people label **every take** without talking about it or looking at each other's labels. You are **annotator A**. A second person (a teammate, or someone Himasara names) is **annotator B**.

```
python label_episodes.py takes/D1_S2_r1                 # annotator A
python label_episodes.py takes/D1_S2_r1 --annotator B    # annotator B
```

A short clip plays for each detected gesture, with one person in a **red box**. Decide about **that person only**:

Key	When
`r`	That person was gesturing **to the dog** (facing or looking at the dog, as the script said)

Key	When
`n`	Not meant for the dog: gesturing to another person, waving to someone off-camera, or a hand shape that happened while talking or scratching
`x`	Bad detection: the red box isn't on a person, there's no visible hand, or it's clearly a software mistake
`SPACE`	Replay the clip
`b`	Go back to the previous one
`q`	Save and quit (you can continue later; it remembers)

Tips: - Use what you know from the script and the take log. In S2, the bystanders' gestures are `n`. - If you can't tell even after replaying, choose `x`. - It takes about 1–2 minutes per take.

To label everything from one day in one go:

```
for t in takes/D1_[SW]*; do python label_episodes.py "$t"; done                # A
for t in takes/D1_[SW]*; do python label_episodes.py "$t" --annotator B; done   # B
```

(Calibration and test takes don't need labels; the check skips them.)

11. Check the dataset (before handing over)

```
python check_dataset.py
```

It shows a table of all takes and a **Problems** list. Fix every problem. Usually it means "run label_episodes.py again to finish labelling". Run it again until it says **"None. Ready to hand over."**

At the end it prints the total number of labelled gestures (target: **300 or more**). If you're well below that, tell Himasara; one more session may be needed.

12. Hand over

1. Copy these to the SSD, in a folder named like `go2_dataset_29Oct`:
2. the whole `~/go2-engagement/engage/takes/` folder,
3. `~/go2-engagement/engage/dataset_report.md`,
4. photos of the **take log** pages.
5. If space allows, also upload the same folder to the shared Google Drive folder Himasara gives you, as a backup.
6. Give Himasara the **paper consent forms and the participant sheet**. They contain names, so they never go into files or Drive.
7. If Claude Code changed any code for you (section 13), push it: `bash cd ~/go2-engagement git add -A git commit -m "data collection fixes" git push -u origin data-collection` Then tell Himasara so she can review it.
8. Send Himasara a message: number of takes, total labelled gestures (from `check_dataset.py`), and anything that went wrong.

13. When something goes wrong

Problem	Fix
`Orbbec camera problem: No device found`	Use a **blue USB-3 port**, re-plug the cable, and try another cable. If setup ran before you plugged it in, re-plug it once.
Camera found but `no frames`	Close any other program using the camera; re-plug.
`NO DEPTH` on screen	Re-plug the camera; run `python check_orbbec.py` and check the right side shows depth.
A person has no box	Move them 0.5 m closer, into more light, away from the picture edge and not behind someone. At most **6 people** are boxed at once; keep it to 5.
Very slow (fps below 8)	Plug in the charger and close other programs.
`already exists`	You reused a take name. Use the next `_r` number.
The window froze	Press `Ctrl+C` in the terminal and record the take again with the next `_r` number.
Anything else	Use Claude Code (below).

Claude Code: open a terminal in `~/go2-engagement` and run `claude`. It reads the project notes automatically. Paste the exact error and say what you were doing. Do **not** let it change the files in `engage/engage/` (`fuzzy.py`, `methods.py`, `perception.py`) or any thresholds; tell it "data collection only, don't change the method".